

CMPT 479/982: GPU Architecture & Programming

Fall 2026

Course Introduction & Simple CUDA Programs

Alaa Alameldeen (alaa@sfu.ca)

When and Where?

- **When:**
 - Wednesdays 9:30 AM - 12:20 PM
- **Where:** ASB 9896
- **Office hours:** After class
- **TA:** Mohammad Taghizadeh Givari
 - Office hours: TBA
- **Webpage:** <https://sfu.instructure.com/courses/19137>
- **Webpage updated frequently:**
 - Class slides
 - Homework and project assignments
 - Changes in class schedule

Course Logistics

- **Delivery:**

- Live lectures in person, will be (hopefully) recorded and posted online
 - ❑ Slides will be posted online, but could change (don't print before class)
- Office hours in person by default but will move to zoom sometimes
- Exams will be in person during class time

- **Communications:**

- Feel free to ask questions during class
- Announcements sent on Piazza
- Participate in office hours
- Discussions on Piazza
 - ❑ Ask course-related questions there since many people might have similar questions
 - ❑ Other students can answer. Instructor and TAs will monitor discussions and answer questions.
- Use email only for urgent questions or concerns.
 - ❑ Email me at alaa@sfu.ca and/or TA (mta188@sfu.ca)

About the Course

- **GPU Architecture and Programming**

- Introduction to the GPU compute and memory architecture
- Programming GPUs using CUDA

- **Topics (tentative):**

- Heterogeneous data-parallel computing using CUDA
- Architecture of Modern GPUs including threads, warps, synchronization, SIMD hardware, thread block and warp scheduling, control divergence
- GPU memory hierarchy and architecture
- GPU architecture performance optimizations
- Examples of CUDA programs and optimizations for convolution, reduction, sorting, and matrix multiplication.
- Time-permitting: Convolutional neural networks, LLMs, dynamic programming, sparse matrix computations

Course Goals

- **Subject:** Theory and practice of GPU architecture and programming
- **Goal:** Learn how to use massively parallel architectures to solve large computational problems
- **Learning Outcomes:**
 - Learn the basics of GPU compute and memory architecture
 - Ability to write programs for GPUs using CUDA
 - Ability to read, review and present research papers
 - Collaborate on a large programming project using CUDA

Expected Background

- **Computer Organization**
 - CMPT 295 OR equivalent
- **Parallel Programming**
 - CMPT 431 OR equivalent
- **Data Structures & Algorithms**
 - CMPT 225 or equivalent
- **Programming Experience in C/C++**

Why take this course?

- Learn how to use GPUs to solve large computational problems
- Understand the architecture of GPUs to help optimize GPU programs
- Improve your programming skills in C/C++, and learn CUDA extensions
- **Warning: Don't take this course if**
 - “I want a course with a light workload”
 - Little experience in C/C++
 - Primary motivation is to get a good grade
 - Can't attend exams during class time
 - “Just want to study from lecture notes – Can't attend class or read textbook”

Grading

- **Grade Breakdown**

- Exams & Quizzes: 40%

- ☐ Three in-person exams each 45-minutes at the start of class time (Worth 14%, 14%, 12% of final grade). **No midterm or final exams**

- Homework assignments: 20%

- ☐ Mostly programming assignments with increasing difficulty
 - ☐ Done individually (no collaboration)
 - ☐ First HW is an online quiz to test your pre-requisite knowledge.

- Project: 40%

- ☐ Group project
 - ☐ 2-3 members per group
 - ☐ Use course discussion boards on Piazza to form groups

- Class participation encouraged (classroom, Piazza)

- ☐ Potential bonus points for top participants. Would help if you are near a grade boundary.

Grading Guidelines

- **Grade Scale (won't get curved)**

- | | |
|----------------------|----------------|
| ➤ A+: 96% and higher | ➤ C+: 65-69.9% |
| ➤ A: 90-95.9% | ➤ C: 60-64.9% |
| ➤ A-: 85-89.9% | ➤ C-: 55-59.9% |
| ➤ B+: 80-84.9% | ➤ D: 50-54.9% |
| ➤ B: 75-79.9% | ➤ F: Below 50% |
| ➤ B-: 70-74.9% | |

- ✓ **Students need 50% or higher in total exam score to pass the course**
- ✓ **Assignments need to run correctly to receive any grade > 0**
 - 0 grade if your code fails to compile OR if it produces wrong results
 - Grading done on CSIL GPU machines. Make sure your code compiles and runs there
- ✓ **Late Policy: You get a total of 5 late days for assignment (EXCEPT HW1 AND PROJECT)**
 - ✓ 1% penalty (from course total) for each late day after the fifth

Important Dates

- **Exam dates (at the beginning of class time):**
 - 07-Oct, 04-Nov, 02-Dec
 - In Person
- **Assignment due dates (tentative, all at 11:59 PM)**
 - 14-Sep, 22-Sep, 05-Oct, 20-Oct
 - Please start early on assignments
- **Project due date: 07-Dec (last day of classes)**
- **No class on:**
 - 30-Sep, 11-Nov (SFU Holidays)
- **Last lecture: 02-Dec**
- **Check course webpage for any schedule changes**

Textbook and Readings

- Reading material (if needed) will be linked off course webpage
- **Textbook [PMPP]:** Wen-mei W. Hwu, David B. Kirk, Izzat El Hajj, "Programming Massively Parallel Processors: A Hands-On Approach," 5th Edition, Morgan-Kaufmann, 2026. ISBN: 978-0443439001
 - We'll go through the first ~14 chapters then a few later ones, time-permitting.
 - Many figures in slides are from the textbook
- **Reference [CPG]:** NVIDIA Corp., "CUDA C++ Programming Guide, Release 13.3," Available online: https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf
- **Other References**
 - Will be linked off the course webpage.
- **Lecture slides alone are not sufficient to study. They are intended to guide the discussion in class not as a standalone resource.**

Academic Integrity

- **Do not cheat!**
 - No sharing of code or solutions
 - Penalties may including getting 0 points on the assignment and/or more severe penalties (suspension or expulsion)
- **Do not post your code on a **public** code repository**
 - Even after course is over
 - You can use private repos: <https://coursys.sfu.ca/2021fa-cmpt-470-d1/pages/GitLab>
- **OK to post project code on public repository if all group members agree.**
- **Homework assignments must be your own work.**
 - Be sure to provide proper citations
 - Discussion ok on Piazza, but no sharing of solutions
 - Only project group members can collaborate on a project.
- **See SFU policies: <https://www.sfu.ca/policies/gazette/student.html>**

Academic Integrity – AI Policy

- **General Advice:** What you learn is proportional to the effort you put in
- **I strongly recommend not using AI to write code for assignments**
- **If you use AI (at your own risk), you need to follow these two rules:**
 1. Acknowledge AI use and include details of AI model used and ALL your prompts in your report.
 2. You are responsible for understanding the AI-generated code. You might be tested on all your self- or AI-generated code in class or in oral exams.
- **AI NOT PERMITTED in Quizzes or Exams.**
- **AI is permitted for coding in your project, but use it at your own risk**
 - Need to acknowledge AI use and include details (and all prompts), similar to assignments
 - Your project report will be graded based on demonstrating your understanding of the project work. If report is poorly-written AI slop, or if it doesn't explain your work in a human-understandable style, this will lower your project grade

Introduction to Parallel Architectures and Programming

(Recap from CMPT 431, LLNL Parallel Computing Tutorial)

Sequential v/s Parallel Execution

Sequential
(single stream of instructions)

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

Parallel
(multiple independent instruction streams)

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

OP [operand, ...]

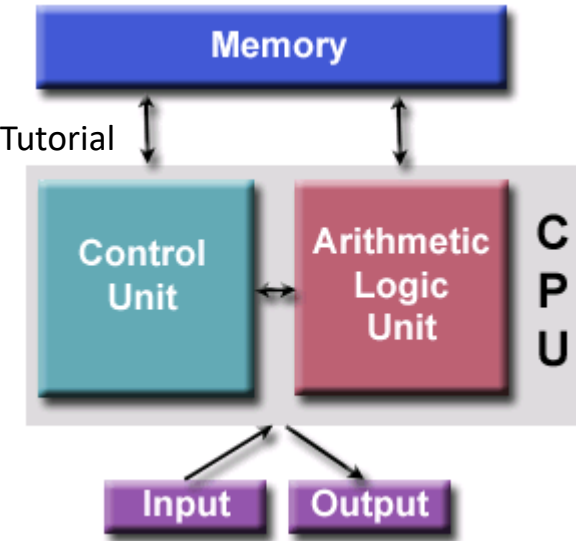
OP [operand, ...]

von Neumann Architecture

- **Uses the stored-program concept**

- The CPU executes a stored program that specifies a sequence of memory read and write operations

Courtesy: LLNL Intro to Parallel Computing Tutorial



- **Basic design:**

- Memory stores both program and data
- Program instructions are coded data which instruct the computer on what to do
- Data: Information to be used by the program
- CPU gets instructions & data from memory, decodes instructions and then performs them sequentially

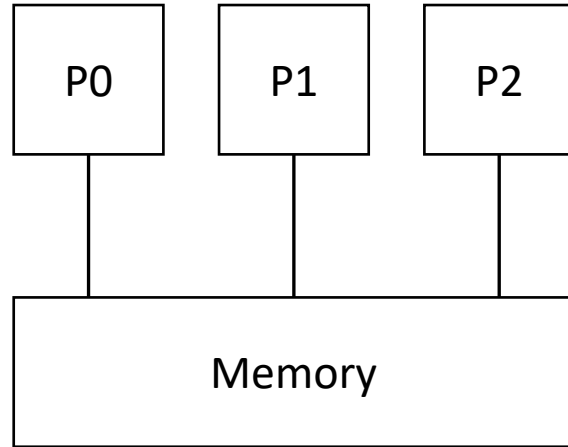
Multiprocessor Taxonomy (Flynn)

S I S D Single Instruction stream Single Data stream	S I M D Single Instruction stream Multiple Data stream
M I S D Multiple Instruction stream Single Data stream	M I M D Multiple Instruction stream Multiple Data stream

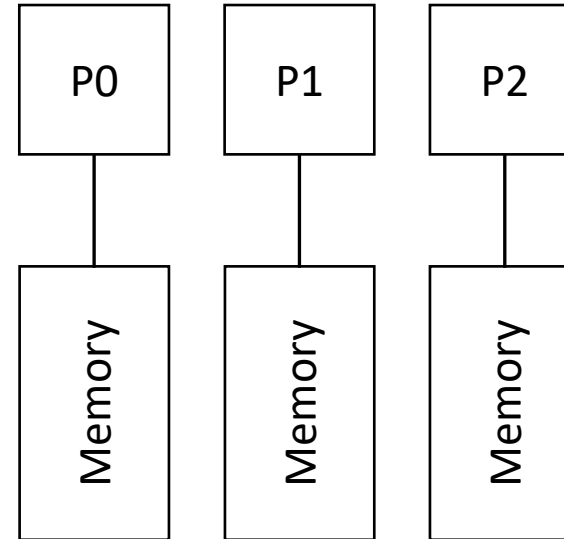
- Instructions and Data streams can be either single or multiple
- **Single Instruction, Single Data (SISD)**
 - Serial, non-parallel computer – e.g., single CPU PCs, workstations and mainframes
- **Single Instruction, Multiple Data (SIMD)**
 - All processor units execute same instruction on different data
 - Example: Vector processors such as IBM 9000, Cray C90
- **Multiple Instruction, Single Data (MISD)**
 - Rare (e.g., C.mmp), data operated on by multiple instructions
- **Multiple Instruction, Multiple Data (MIMD)**
 - **Most modern parallel computers, including GPUs**
 - Processors may be executing different instructions on different data streams

Memory Architectures

Shared Memory



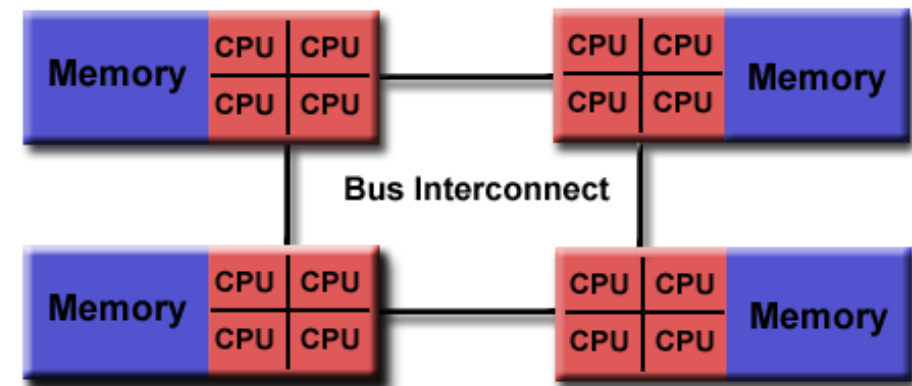
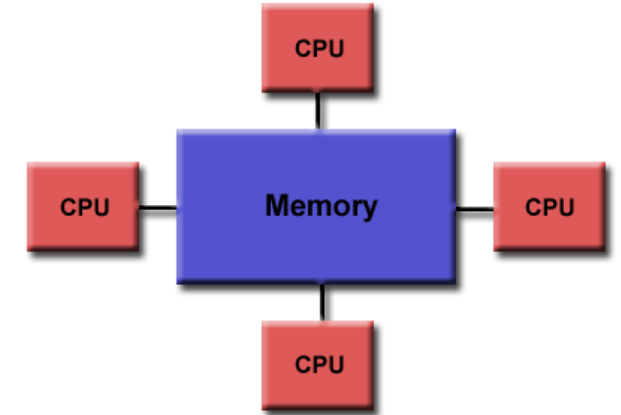
Distributed Memory



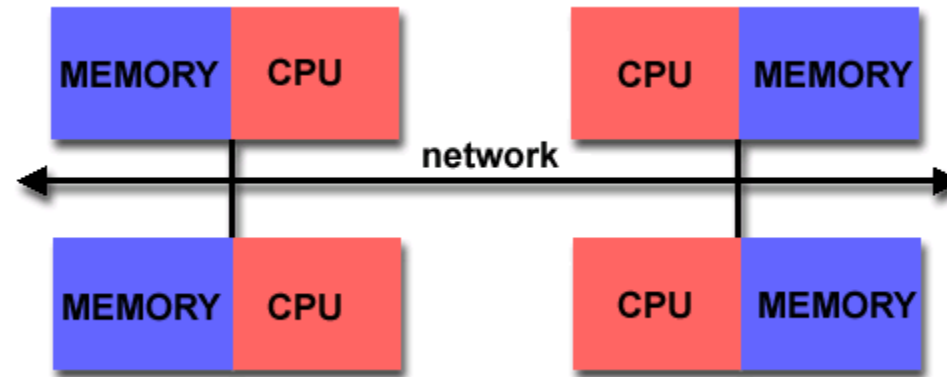
Shared Memory Architecture

Courtesy: LLNL Intro to Parallel Computing Tutorial

- All processors can access all memory
- Processors share memory resources, but can operate independently
- One processor's memory changes are seen by all other processors
- **Uniform Memory Access (UMA) Architecture**
 - Example: Symmetric Multiprocessor (SMP) machines
 - Identical processors with equal access and equal access time to memory
 - Also called CC-UMA - Cache Coherent UMA. Cache coherent means that if one processor updates a location in shared memory, all the other processors know about the update
- **Non-Uniform Memory Access (NUMA) Architecture**
 - Often made by physically linking two or more SMPs
 - One processor can directly access memory of another process
 - Not all processors have equal access time to all memories
 - Memory access across link is slower
 - Called CC-NUMA - Cache Coherent NUMA - if CC is maintained



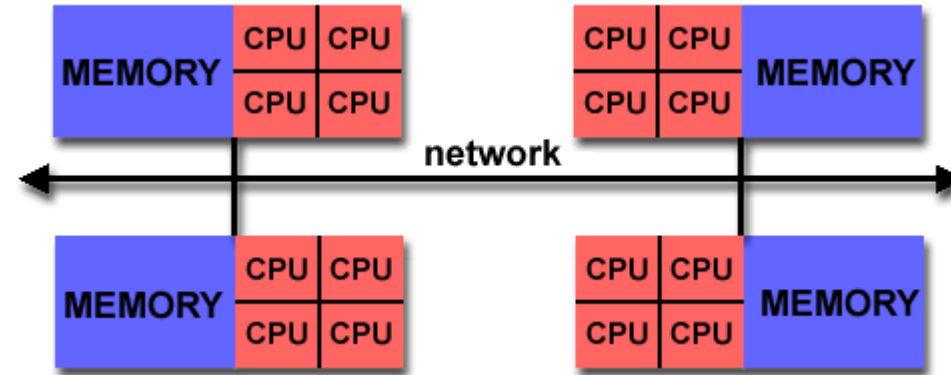
Distributed Memory Architecture



Courtesy: LLNL Intro to Parallel Computing Tutorial

- **Require a communication network to connect inter-processor memory**
- **Processors have their own local memory**
 - Memory addresses in one processor do not map to another processor
 - No concept of global address space across all processors.
- **Each processor operates independently, changes to its local memory have no effect on the memory of other processors**
 - Cache coherence does not apply
- **Programs explicitly define how and when data is communicated and how tasks are synchronized**

Hybrid Distributed-Shared Memory Architecture



Courtesy: LLNL Intro to Parallel Computing Tutorial

- **Shared memory component is usually a CC SMP**
 - Processors on a given SMP can address that machine's memory as global memory
- **Distributed memory component is the networking of multiple SMPs**
 - SMPs know only about their own memory
 - Network communications needed to move data between SMPs
- **Largest, fastest computers share this architecture**

Sequential vs. Parallel Execution

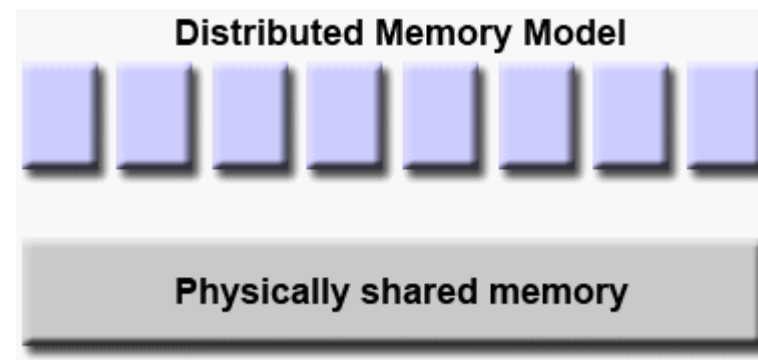
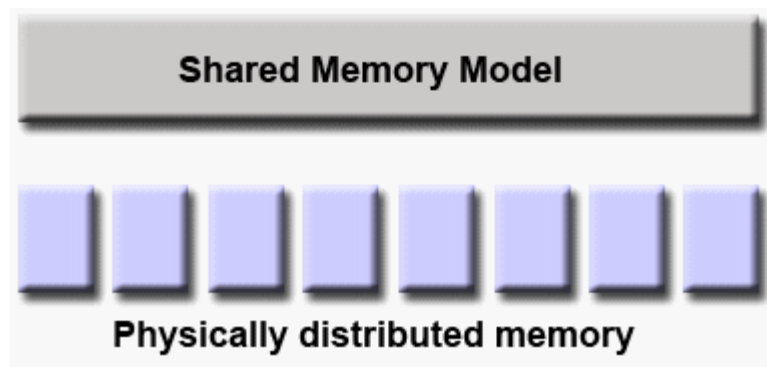
```
1  withdraw (int account_id, int amount) {  
2    balance = get_balance(account_id);  
3    if (balance >= amount) {  
4      set_balance (account_id, balance - amount);  
5      eject(amount);  
6    }  
7  }
```

withdraw(5, 900);	withdraw(5, 700);
2 balance = 1000	2 balance = 1000
3 balance >= 900	3 balance >= 700
4 balance = 100	4 balance = -600

Time
↓

Parallel Programming Models

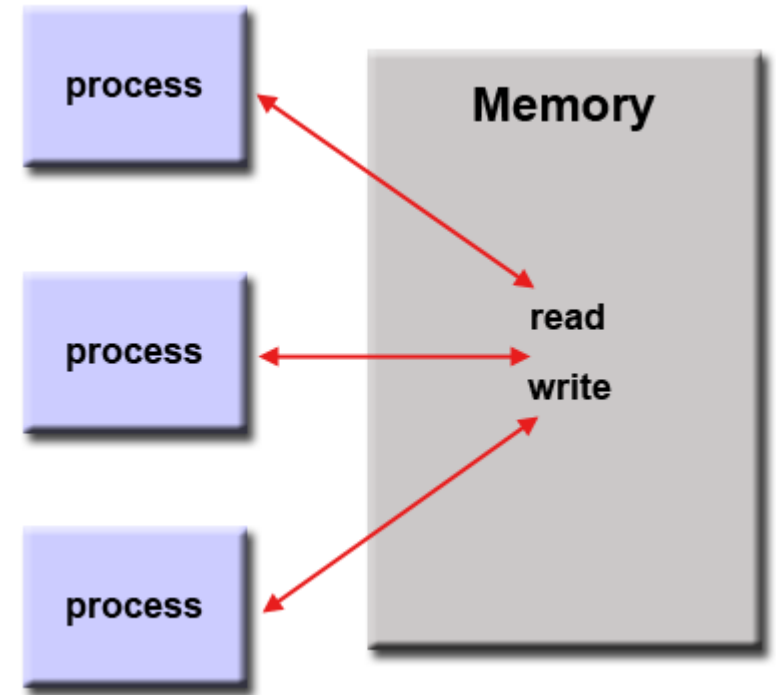
- **Parallel programming models:**
 - Shared Memory
 - Threads
 - Message Passing
 - Data Parallel
 - Hybrid
- **Parallel programming models exist as an abstraction above hardware and memory architectures**
- **Models not specific to certain types of memory architecture or machine**
 - Shared memory can be implemented on a distributed memory architecture
 - Message passing can be implemented on a shared memory architecture (e.g., SGI Origin)



Courtesy: LLNL Intro to
Parallel Computing
Tutorial

Shared Memory Model

- Tasks share a common address space, which they read and write asynchronously
- Various mechanisms such as locks / semaphores may be used to control access to the shared memory
- **Relatively simple programming model**
 - No notion of data "ownership"
 - No need to specify explicit communication of data between tasks
- **But it's more difficult to understand and manage data locality**
 - We don't know OR we can't take advantage of memory being physically closer to a particular process

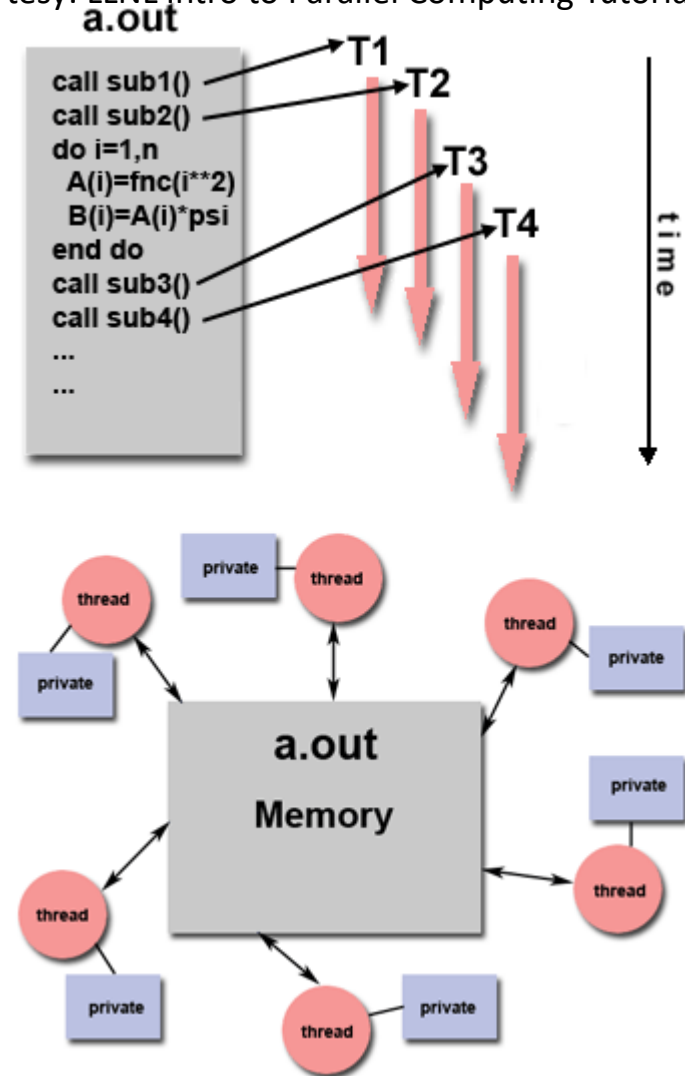


Courtesy: LLNL Intro to Parallel Computing Tutorial

Threads Model

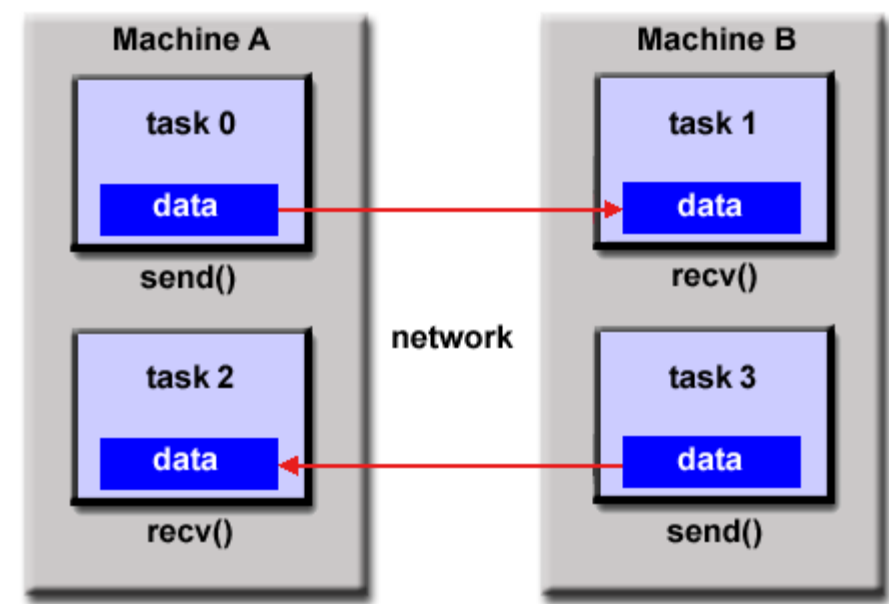
- Single process can have multiple, concurrent execution paths
- **Analogy: a single program that includes a number of subroutines**
 - Program creates a number of tasks (threads) that can be scheduled and run by the operating system concurrently
- Each thread has local data, but also shares all resources of the program (e.g., memory space)
- Threads communicate with each other through global memory by updating address locations
 - This requires synchronization constructs to ensure that more than one thread is not updating the same global address at the same time
- Threads are commonly associated with shared memory architectures and operating systems
- Implementation standards
 - POSIX threads (Pthreads): Library-based, available in C
 - ❑ Tutorial: <https://computing.llnl.gov/tutorials/pthreads/>
 - OpenMP: Compiler-directive-based
 - ❑ Tutorial: <https://computing.llnl.gov/tutorials/openMP/>

Courtesy: LLNL Intro to Parallel Computing Tutorial



Message Passing Model

- Tasks use their own local memory during computation
- Multiple tasks can run on the same physical machine OR across an arbitrary number of machines
- Tasks exchange data through communications by sending and receiving messages

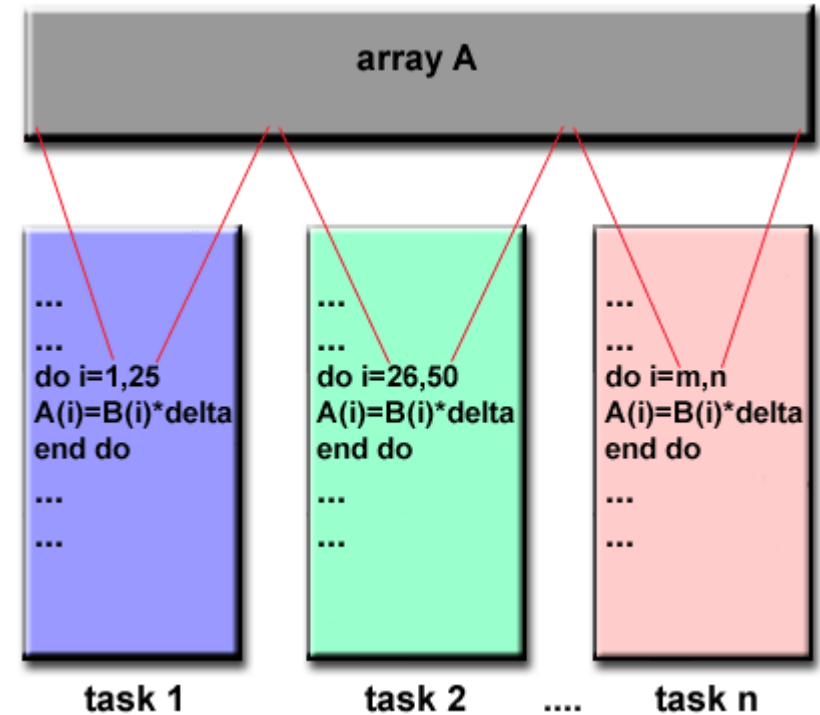


Courtesy: LLNL Intro to Parallel Computing Tutorial

- Requires cooperative operations by each process, e.g., a send operation must have a matching receive operation
- **Implementations**
 - Library of subroutines that are embedded in source code. The programmer is responsible for determining all parallelism.
 - Message Passing Interface (MPI) is the current industry standard for message passing
 - ❑ Tutorial: <https://computing.llnl.gov/tutorials/mpi/>
 - For shared memory architectures, MPI implementations use shared memory (memory copies) for task communications

Data Parallel Model

- Most parallel work focuses on performing operations on a data set
 - Data set is typically organized into a common structure, such as an array or cube
- Each task works on a different partition of the same data structure, all tasks work collectively on the structure
- Tasks perform the same operation on their partition of work, e.g., "add 4 to every array element"
- On shared memory architectures, all tasks may have access to the data structure through global memory
- On distributed memory architectures the data structure is split up and resides as "chunks" in the local memory of each task
- On GPUs, each thread operates on its own share of data



Courtesy: LLNL Intro to Parallel Computing Tutorial

Heterogeneous Parallel Computing

Microprocessor Design: Multi-Core vs. Many-Thread

- **Multi-Core:**

- **Focus on latency of sequential programs.**
- Maintains execution speed of sequential programs via wide superscalar out of order pipelines
- Scales a single CPU to a few up to hundreds of cores. Number of cores scales each generation.
- Can be heterogeneous, e.g., Big-Little with a few large highly capable cores and many small energy-efficient cores.

- **Many Thread, e.g. GPUs**

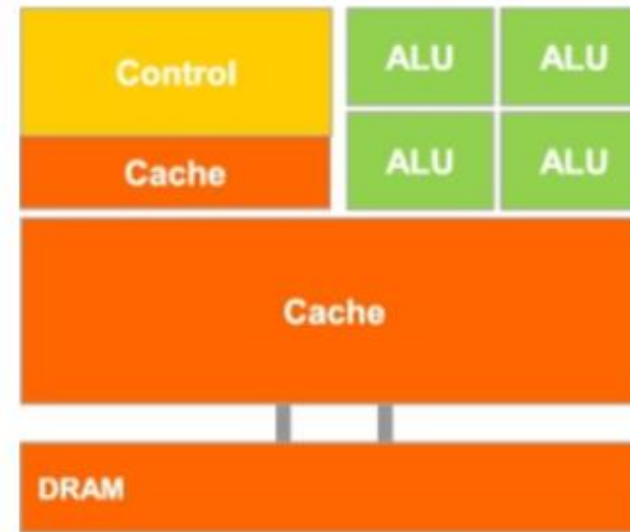
- **Focus on throughput of parallel applications**
- Includes thousands of threads, up to hundreds of thousands in recent GPUs.
- Achieve very high throughput measured in TFLOPS (Tera Floating-point Operations Per Second). Throughput depends on data precision – much higher for half-precision vs. single-precision vs. double-precision

CPU vs. GPU Design Tradeoffs

Courtesy: PMPP Textbook

- **CPU: Latency-oriented**

- Large ALUs to minimize latency of individual operations
- Large multi-level caches to capture spatial and temporal locality
- Complex control with wide superscalar pipelines and dynamic scheduling
- Speculative execution resources with branch prediction, memory dependence prediction, data forwarding
- May include multithreading with 2-4 threads per core



CPU

(latency-oriented design)



GPU

(throughput-oriented design)

FIGURE 1.1 CPUs and GPUs have fundamentally different design philosophies.

CPU vs. GPU Design Tradeoffs

Courtesy: PMPP Textbook

- **GPU: Throughput-oriented**

- Large number of small execution units to run many parallel operations
- Simple control with no speculation or forwarding
- Higher memory bandwidth required to transfer data needed for parallel operations
- Smaller caches to control bandwidth demand
- Cheaper to increase throughput instead of decreasing latency.

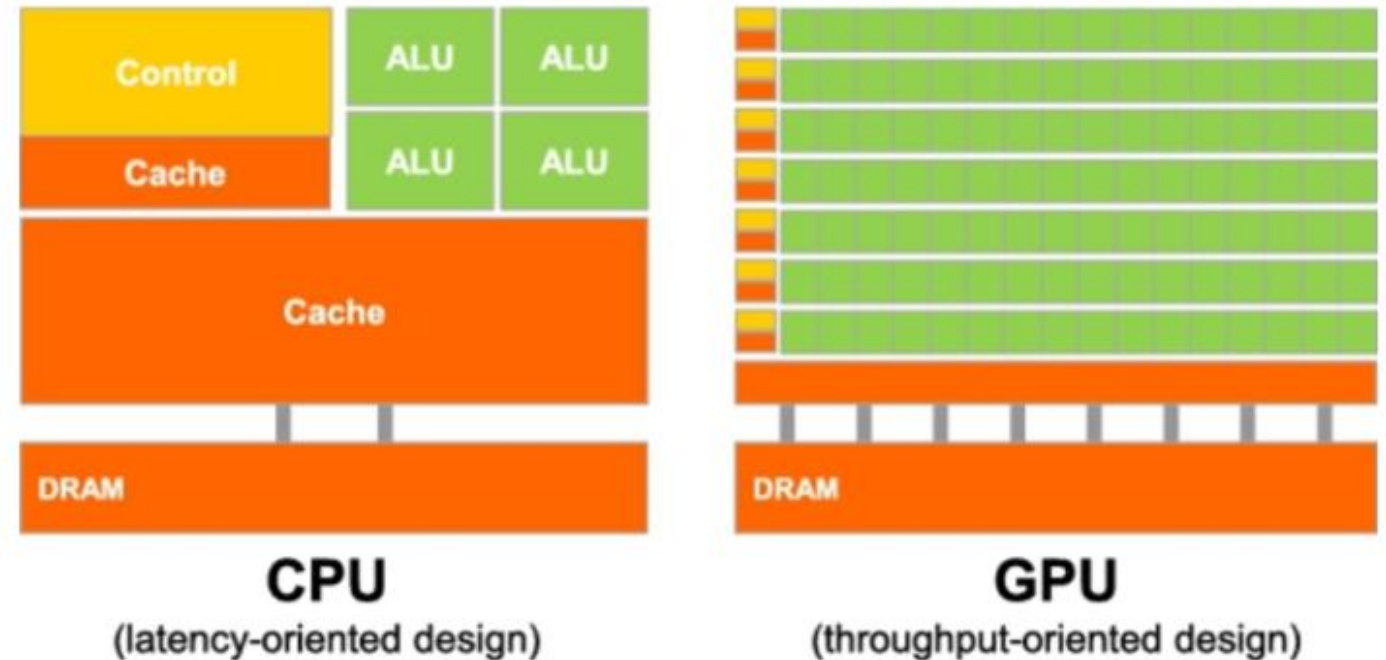


FIGURE 1.1 CPUs and GPUs have fundamentally different design philosophies.

Why GPUs are Becoming More Popular

- Much larger installation base due to graphics and gaming applications (which encourages SW development) vs. limited number of supercomputers.
- Easier to deploy in the market vs. datacenters. Example: MRI machines can have GPUs but not multi-rack servers.
- CUDA and hardware changes by NVIDIA made parallel programming easier. With a general-purpose programming interface, CUDA expanded the types of applications that can run on GPUs.
- Heterogeneous applications run latency-sensitive (sequential) code on CPU, and parallel code on GPU
- Many types of applications are heterogeneous and can benefit from GPU acceleration: Machine Learning, LLMs, Simulation, Data Analytics, Financial Analysis, Medical Imaging, Audio/Video Processing, Vision, Ray Tracing, ...etc.

GPGPU vs. CUDA

- **GPGPU:** General-Purpose programming using a Graphics Processing Unit. Programmers needed to use graphics API functions (e.g., OpenGL, Direct3D) to access the processing units. That is, a computation must be expressed as a function that paints a pixel to execute on GPUs.
- **CUDA:** Compute Unified Device Architecture. Allowed Using GPUs for general-purpose data-parallel programming, not just for pixel computations. Enabled via programming language extensions and hardware changes

Courtesy: PMPP Textbook

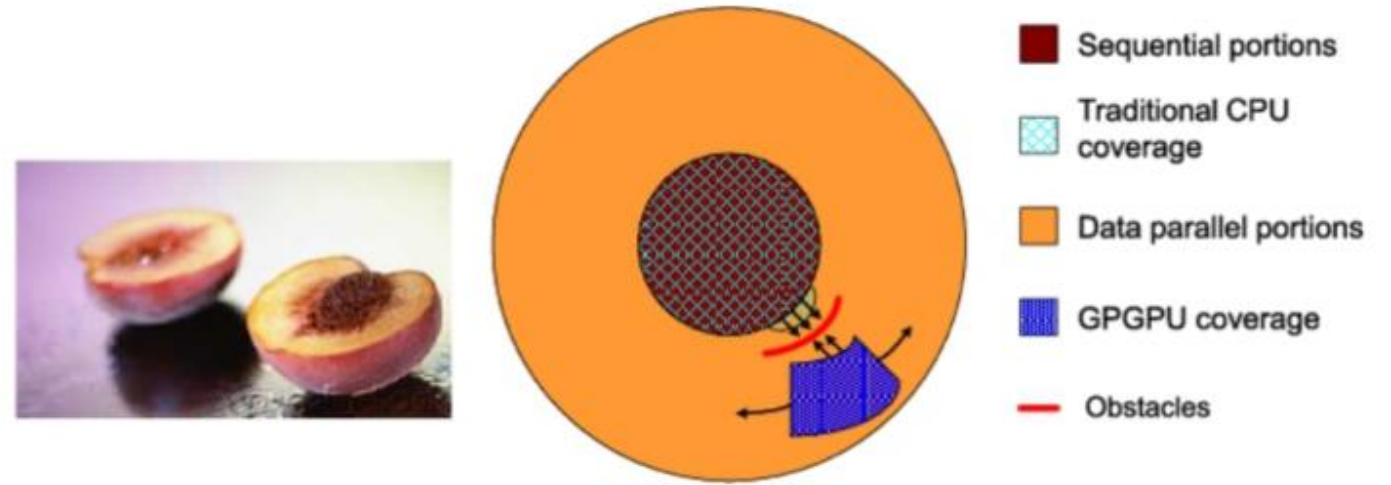


FIGURE 1.2 Coverage of sequential and parallel application portions. The sequential portions and the traditional (single-core) CPU coverage portions overlap with each other. The previous GPGPU technique offers very limited coverage of the data parallel portions since it is limited to computations that can be formulated into painting pixels. The obstacles refer to the power constraints that make it hard to extend single-core CPUs to cover more of the data parallel portions.

Performance Metrics and Limits

Measuring Performance

- **Latency:** How fast does a job compute? Elapsed time
 - Time for computation + communication + synchronization
- **Throughput:** How many jobs complete in a given time?
- **Scalability:** How well does the system scale?
 - **Strong Scaling:** How the solution time varies with the number of processors (or compute nodes) for a *fixed total problem size*?
 - **Weak Scaling:** How the solution time varies with the number of processors (or compute nodes) for a *fixed problem size per processor*?

Weak v/s strong scaling: https://en.wikipedia.org/wiki/Scalability#Weak_versus_strong_scaling

Performance Metrics

- **Speedup**, $S_p = \frac{\text{Execution time on 1-processor system } (T_1)}{\text{Execution time on } p\text{-processor system } (T_p)}$
- **Alternatively**, $S_p = \frac{\text{Throughput on } p \text{ processors } (X_p)}{\text{Throughput on 1 processor } (X_1)}$
- **Efficiency** $= \frac{S_p}{p}$ (higher is better)
- **Cost** $C_p = p \times T_p$ Optimal if $C_p = T_1$

Amdahl's Law [1967, Gene Amdahl]

- F = fraction of problem that is parallel
➤ $(1 - F)$ = fraction of problem that is sequential

- Time on 1 processor = $\frac{1-F}{1} + \frac{F}{1} = 1$

- Time on p processors = $\frac{1-F}{1} + \frac{F}{p}$

- Speedup with p processors:

$$S_p = \frac{1}{1-F + \frac{F}{p}}$$

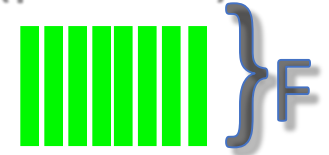
- Fastest parallel time, $T_p = T_1 \times \left(1 - F + \frac{F}{p}\right)$

Program Phases

(Serial)



(parallel)

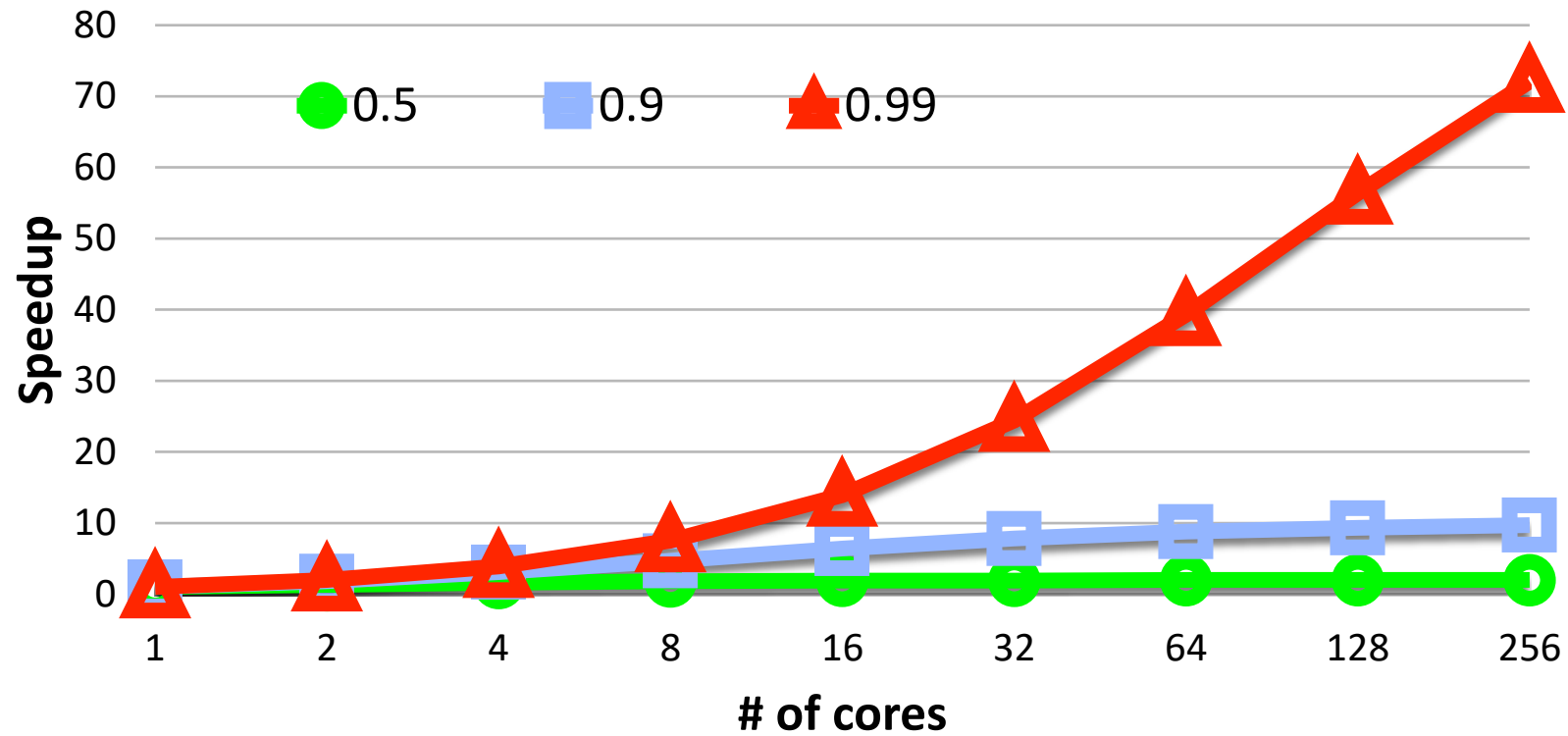


Implications of Amdahl's Law

1. As the parallel part increases, Speedup also increases: $S_p \uparrow$ as $F \uparrow$

- Implies that programmers should focus on parallelizing as much of the program as possible

Parallel Speedup



Implications of Amdahl's Law

1. As the parallel part increases, Speedup also increases: $S_p \uparrow$ as $F \uparrow$

➤ Implies that programmers should focus on parallelizing as much of the program as possible

2. Gives an upper bound on parallel speedup

Only fraction F can be concurrently run by p processors

➤ Increasing p cannot speed up fraction $1-F$

• Speedup with p processors, $S_p = \frac{1}{1-F + \frac{F}{p}}$

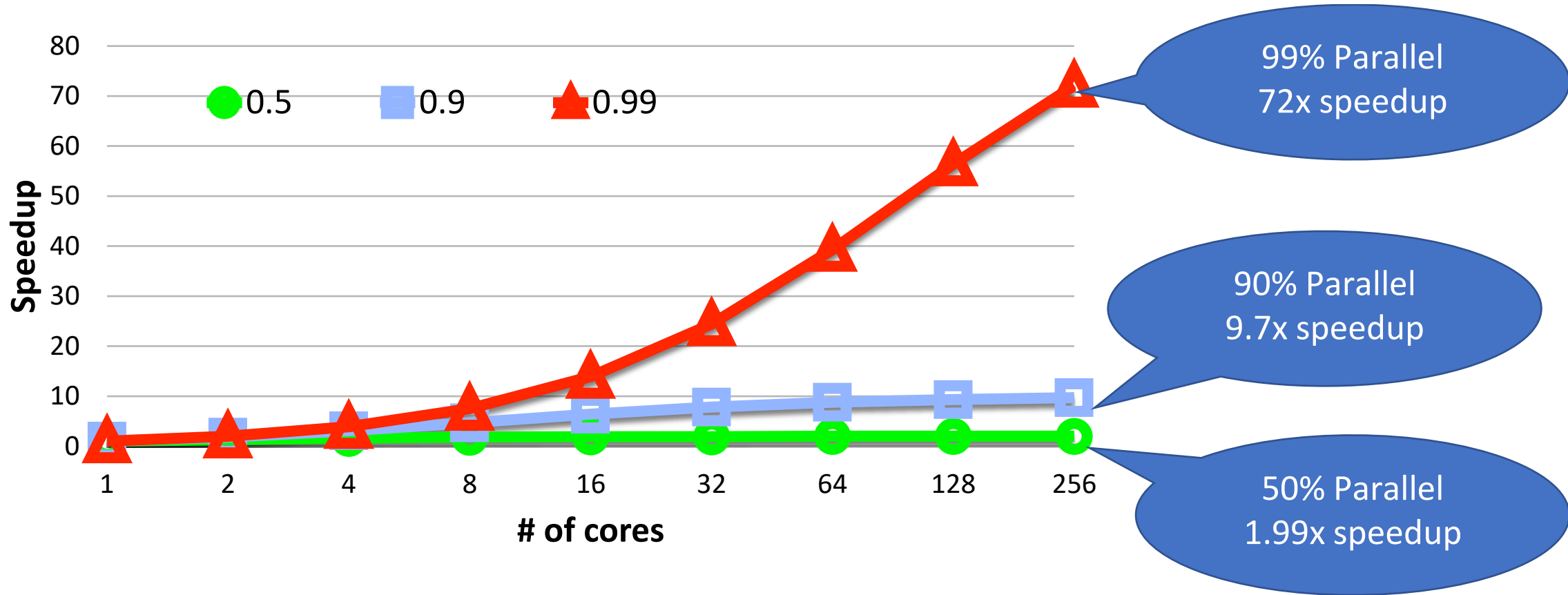
• Upper bound on speedup at $S_\infty = \frac{1}{1-F}$

• Example: If $F = 0.67$:

➤ Speedup for 4 cores = 2

➤ Speedup for ∞ processors = 3

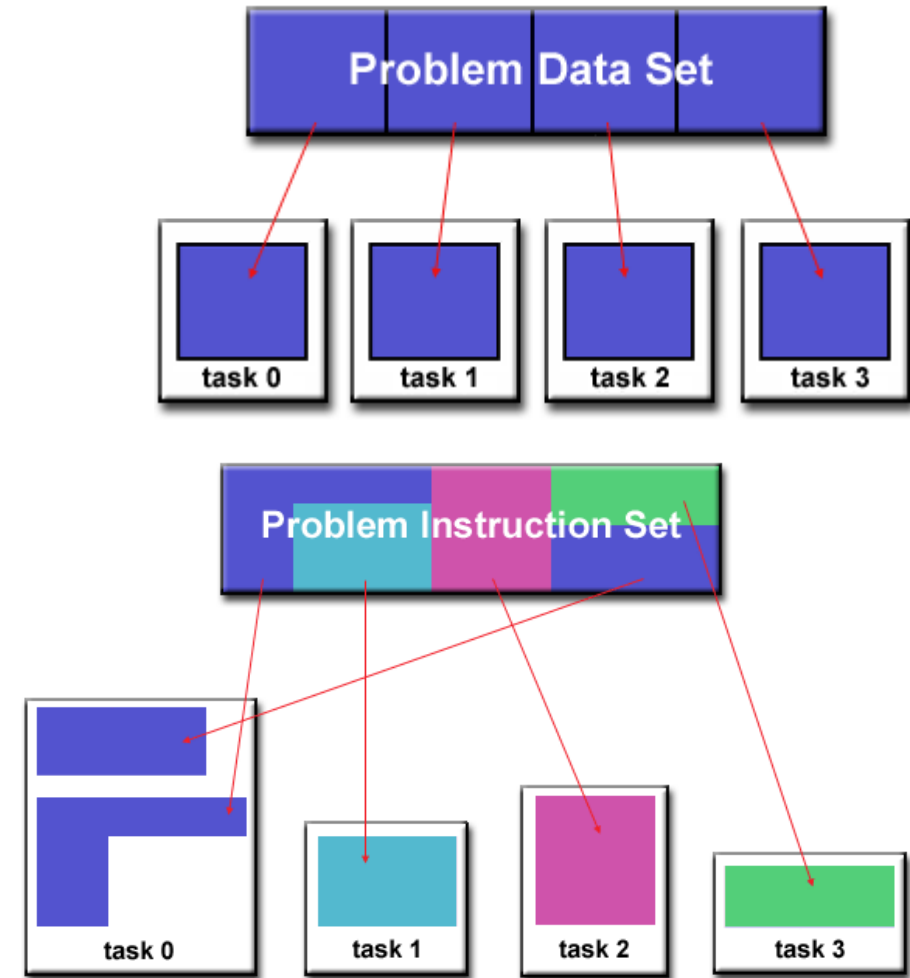
Parallel Speedup



Challenges in Parallel Program Design

Designing Parallel Programs: Partitioning

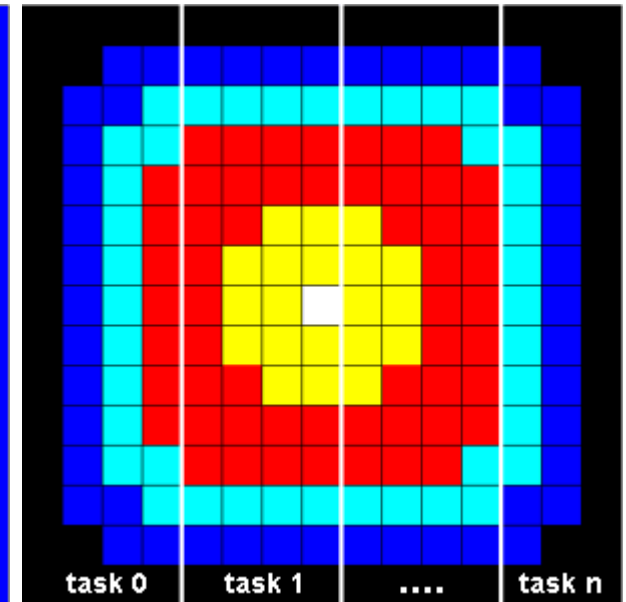
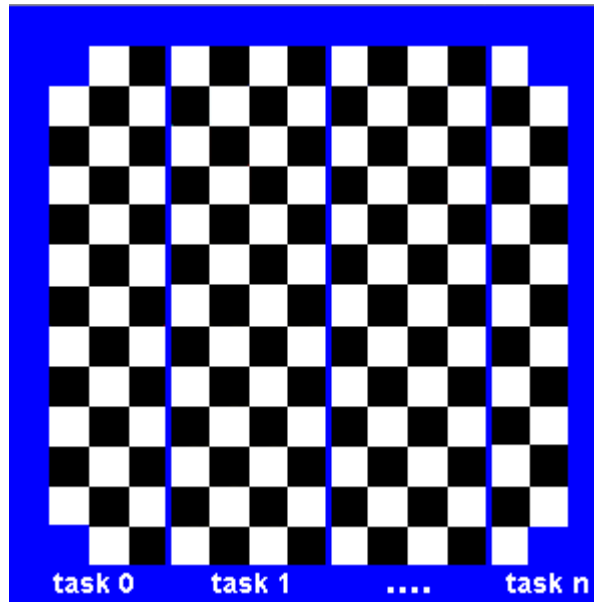
- **Breaking the problem into discrete "chunks" of work that can be distributed to multiple tasks**
 - Two main types: domain decomposition and functional decomposition
- **Domain Decomposition**
 - Data is decomposed, each task works on portion of data
 - Different ways to partition data
- **Functional Decomposition**
 - The problem is decomposed according to the work that must be done, each task performs a portion of the overall work
- **Programmer can combine both types**



Courtesy: LLNL Intro to Parallel Computing Tutorial

Designing Parallel Programs: Communication

- Some problems can be decomposed and executed in parallel with virtually no data sharing, called *embarrassingly parallel* problems
 - Example: image processing operation where every pixel in a black and white image needs to have its color reversed. The image data can easily be distributed to multiple tasks that then act independently of each other to do their portion of the work
- Most parallel applications are not quite so simple, and do require data sharing between tasks
- Issues to consider when writing a program that needs communications
 - Cost of communications
 - Latency vs. bandwidth
 - Visibility of communications
 - Synchronous vs. asynchronous communications
 - Scope of communications
 - Efficiency of communications
 - Overhead and complexity



Courtesy: LLNL Intro to
Parallel Computing Tutorial

Designing Parallel Programs: Synchronization

- **Barrier (all tasks)**

- Each task performs its work until it reaches the barrier then stops (blocks)
- When the last task reaches the barrier, all tasks are synchronized
- Tasks can then continue or serial work is done

- **Lock / semaphore (any number of tasks)**

- Typically used to serialize (protect) access to global data or a section of code
- Only one task at a time may own the lock / semaphore / flag.
- The first task to acquire the lock "sets" it and then it can then safely (serially) access the protected data or code
- Other tasks can attempt to acquire the lock but must wait until the task that owns the lock releases it
- Can be blocking or non-blocking

- **Synchronous communication operations (tasks performing communication operation)**

- Coordination is required with the other task(s) participating in the communication, e.g., before a task can perform a “send” operation, it must receive an acknowledgment from the receiving task that it is OK to send.

Designing Parallel Programs: Data Dependencies

- A *dependence* exists between program statements when the order of statement execution affects the results of the program
- A *data dependence* results from multiple use of the same location(s) in storage by different tasks
- Dependencies are one of the primary **inhibitors** to parallelism
- Loop carried dependencies are particularly important since loops are possibly the most common target of parallelization efforts

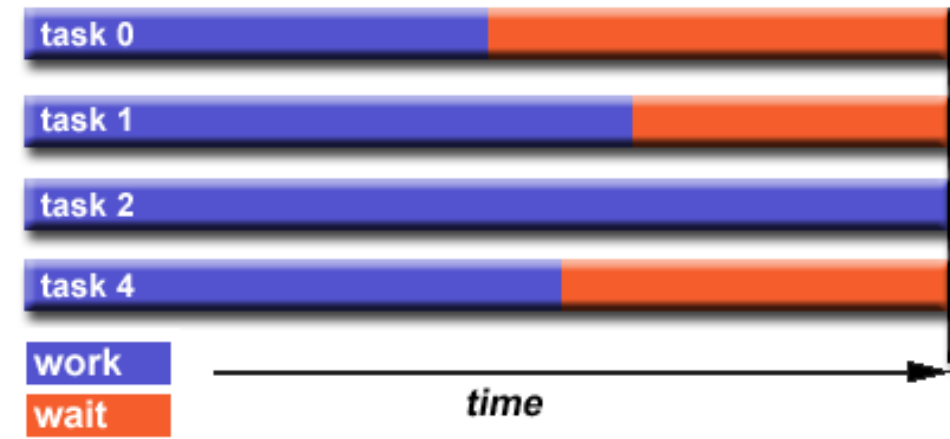
```
for (i = 1; i < N; i++)
```

```
    A[i] = A[i-1]*2;
```

- How to handle data dependencies
 - Distributed memory architectures: communicate required data at synchronization points
 - Shared memory architectures: synchronize read/write operations between tasks

Designing Parallel Programs: Load Balancing

- Load balancing refers to distributing work among tasks so that all tasks are kept busy all the time
 - Minimize idle time
- Important for performance
 - If tasks are synchronized by a barrier, slowest task determines the overall performance
- How to achieve load balancing
 - Equally partition the work each task receives
 - Dynamic work assignment

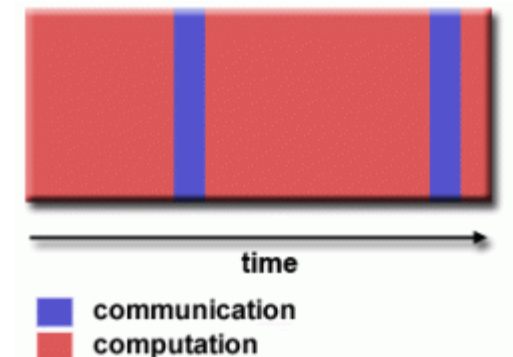
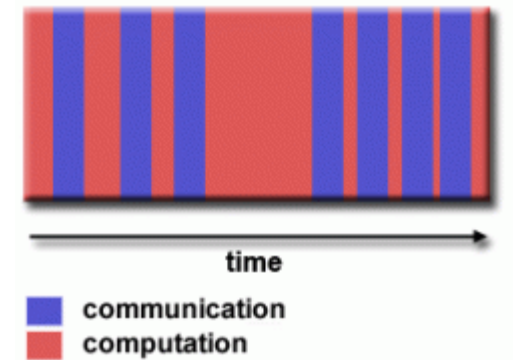


Courtesy: LLNL Intro to Parallel Computing Tutorial

Designing Parallel Programs: Granularity

- Granularity is a qualitative measure of the ratio of computation to communication
- Periods of computation are typically separated from periods of communication by synchronization events.
- **Fine-grain Parallelism**
 - Relatively small amounts of computational work are done between communication events, low computation to communication ratio
 - Facilitates load balancing
 - High communication overhead, less opportunity to improve performance
 - Overhead required for communications and synchronization between tasks can be longer than the computation
- **Coarse-grain Parallelism**
 - Relatively large amounts of computational work are done between communication/synchronization events
 - High computation to communication ratio
 - Implies more opportunity for performance improvement
 - Harder to load balance efficiently

Courtesy: LLNL Intro to Parallel Computing Tutorial



Heterogenous Data-Parallel Computing

Data Parallelism

- **Recall Data Parallel Programming Model:**

- Data is divided between tasks, each task performs similar operations on its share of data
- Data is partitioned among tasks: Domain Decomposition
- Many applications benefit from data parallelism

- **Example: Color to Grayscale Conversion**

- Each pixel in a picture can be individually converted using this formula to compute its luminance value from individual red (r), green (g), blue (b) color values from RGB representation:

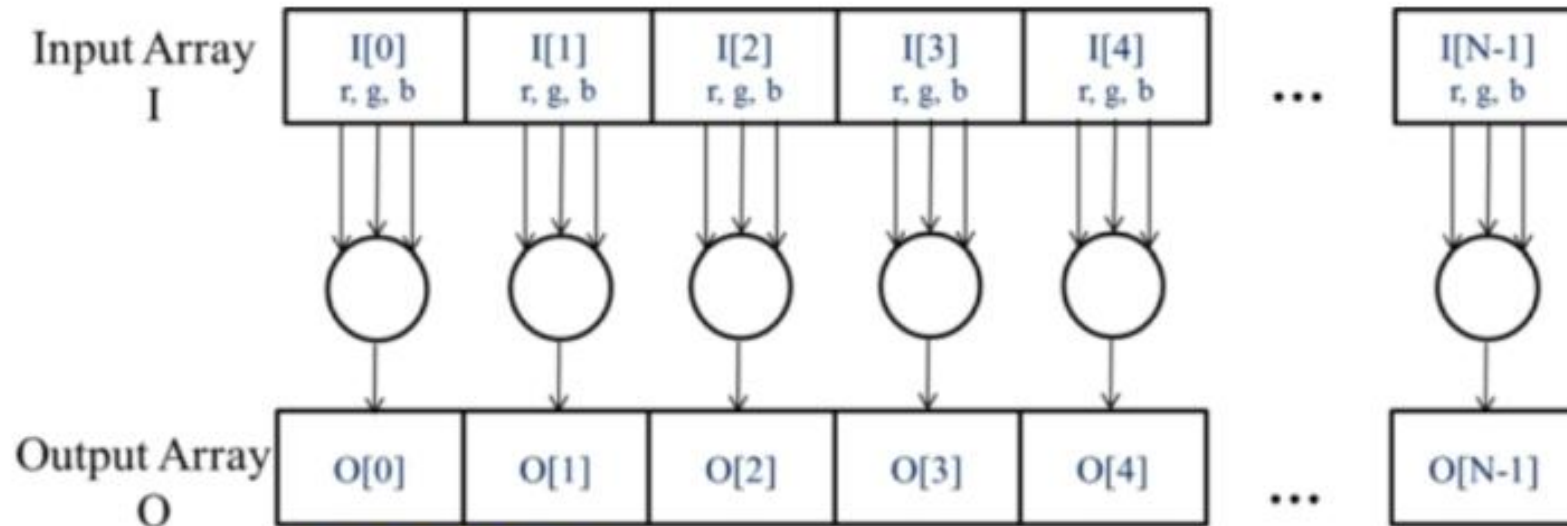
$$L = 0.299 * r + 0.587 * g + 0.114 * b$$

Courtesy: PMPP Textbook



FIGURE 2.1 Conversion of a color image to a grayscale image.

Color to Grayscale Conversion



Courtesy: PMPP Textbook

FIGURE 2.2 Data parallelism in color-image-to-grayscale conversion – pixels can be calculated independently of each other.

- Each pixel in input figure is represented as three bytes in memory, one for each of the R, G, and B values
- Output is a byte for each pixel representing its luminance value.

CUDA C++ Programs

- CUDA C++ extends C++ programming language with minimal new syntax and library functions
 - Enables writing programs for heterogeneous computing systems with both CPUs and GPUs.
- Built on NVIDIA's CUDA platform, the most mature framework for programming massively-parallel systems.
- Compiles programs using the nvcc compiler. Simple command lines:

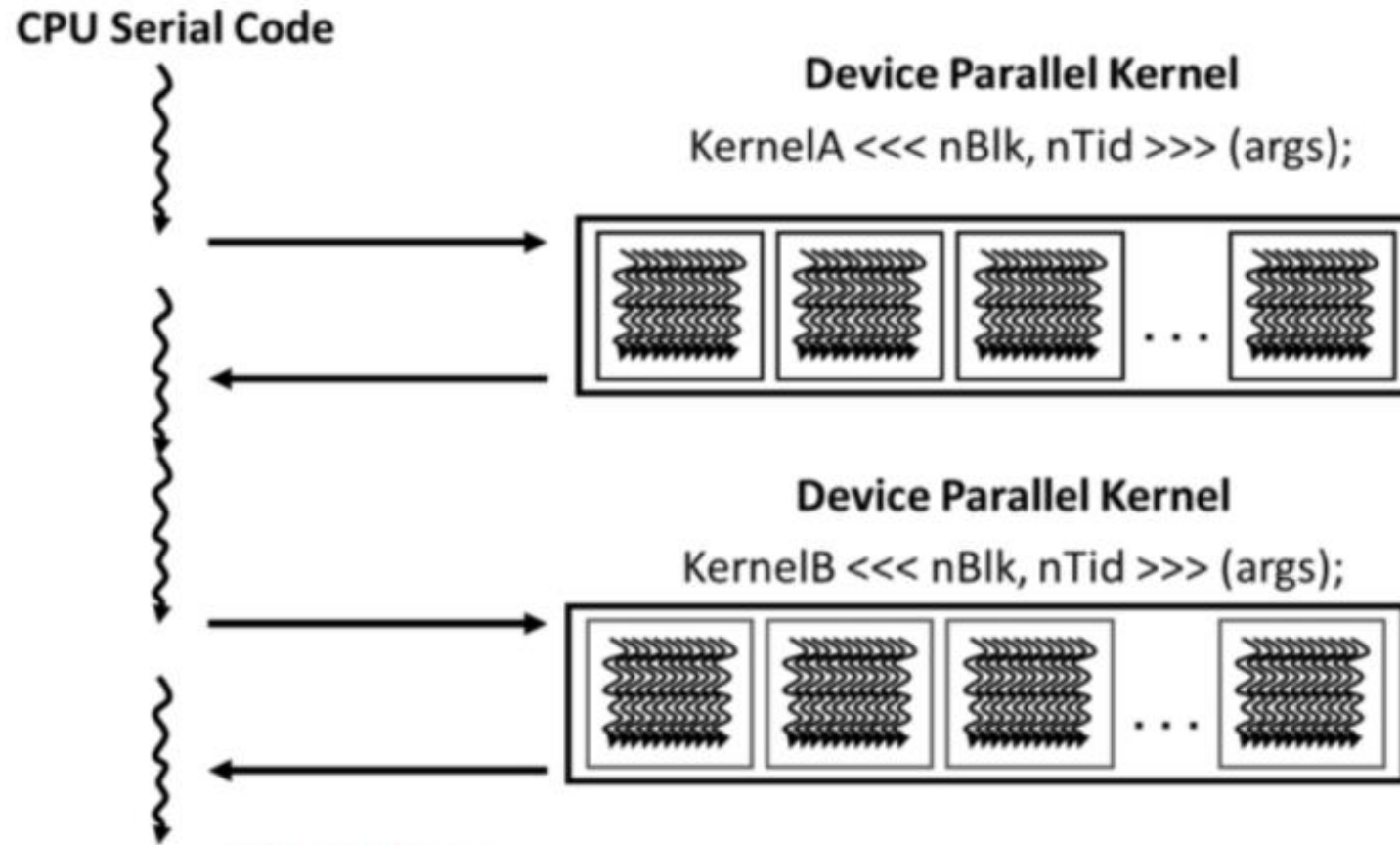
CPU programs:

```
nvcc    myprogram.cpp    -o myprogram
```

Programs with CUDA syntax:

```
nvcc    -x cu    myprogram.cpp    -o myprogram
```

Execution of a CUDA Program on CPU & GPU



Courtesy: PMPP Textbook

FIGURE 2.3 Execution of a CUDA program.

- Ideally, all sequential code runs on CPU, and all data parallel code runs on GPUs as “device parallel kernels”

Simple Example: Vector Addition

Courtesy: PMPP Textbook

```
01 // Compute vector sum C_h = A_h + B_h
02 void vecAdd(float* A_h, float* B_h, float* C_h, int n) {
03     for (int i = 0; i < n; ++i) {
04         C_h[i] = A_h[i] + B_h[i];
05     }
06 }
07 int main() {
08     // Memory allocation for arrays A, B, and C
09     // I/O to read A and B, N elements each
10     ...
11     vecAdd(A, B, C, N);
12 }
```

FIGURE 2.4 A simple traditional vector addition C++ code example.

- Adds corresponding elements of two input vectors A and B; and stores the output in vector C
- Above is the sequential version. Corresponding elements are added sequentially in order

Vector Addition: How to Parallelize?

Courtesy: PMPP Textbook

- Move the actual addition to the “device”, i.e. GPU to run in parallel
- Vectors A, B, and C are allocated in host memory.
- A & B are copied to device memory which computes C
- C is then copied back to host memory.
- Variable names `_h` for host, `_d` for device memory allocation

```
01 void vecAdd(float* A, float* B, float* C, int n) {
02     int size = n* sizeof(float);
03     float *d_A *d_B, *d_C;
04
05     // Part 1: Allocate device memory for A, B, and C
06     // Copy A and B to device memory
07     ...
08
09     // Part 2: Call kernel - to launch a grid of threads
10     // to perform the actual vector addition
11     ...
12
13     // Part 3: Copy C from the device memory
14     // Free device vectors
15     ...
16 }
```

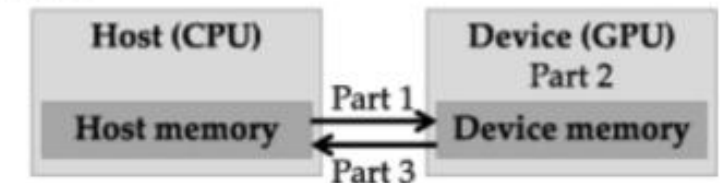


FIGURE 2.5 Outline of a revised `vecAdd` (host code) function that moves the work to a device.

Device Global Memory Functions

Courtesy: PMPP Textbook

- GPUs have their own device memory that is available for programmers
- GPU DRAM is called device “global memory”, or just “global memory”
- `cudaMalloc()` and `cudaFree()` allocate and free device memory
- `CudaMemcpy()` transfers between host and device memory.

`cudaMalloc()`: allocates object in the device global memory and requires two parameters

- **Address of a pointer** to the allocated object
- **Size** of allocated object in terms of bytes

`cudaFree()`: frees object from device global memory and requires one parameter

- **Pointer** to freed object

FIGURE 2.6 CUDA API functions for managing device global memory.

`cudaMemcpy()`: performs memory data transfer and requires four parameters

- **Pointer** to destination
- **Pointer** to source
- **Size** in number of bytes copied
- **Type/Direction** of transfer

FIGURE 2.7 CUDA API function for data transfer between host and device.

Vector Addition with Allocation & Data Transfer

Courtesy: PMPP Textbook

```
01 void vecAdd(float* A_h, float* B_h, float* C_h, int n) {  
02     int size = n * sizeof(float);  
03     float *A_d, *B_d, *C_d;  
04  
05     cudaMalloc((void **) &A_d, size);  
06     cudaMalloc((void **) &B_d, size);  
07     cudaMalloc((void **) &C_d, size);  
08  
09     cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);  
10     cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);  
11  
12     // kernel invocation code - to be shown later  
13     ...  
14  
15     cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);  
16  
17     cudaFree(A_d);  
18     cudaFree(B_d);  
19     cudaFree(C_d);  
20 }
```

FIGURE 2.8 A more complete version of the vecAdd function.

- Includes allocation, freeing and copying data to/from device memory.
- Host memory allocation and free operations are omitted.

Note Regarding Error Handling

- CUDA functions can fail, and programs many need to handle errors
 - For example: cudaMalloc() may fail because device is out of memory
- CUDA programs can use a cudaError type return value to identify whether an error has occurred. This can save debug time. Example:

```
cudaError_t err = cudaMalloc((void **) &A_d, size);  
if (error != cudaSuccess) {  
    printf("%s in %s at line %d \n",  
           cudaGetErrorString(err), __FILE__, __LINE__);  
    exit(EXIT_FAILURE);  
}
```

Kernel Functions and Threading

Courtesy: PMPP Textbook

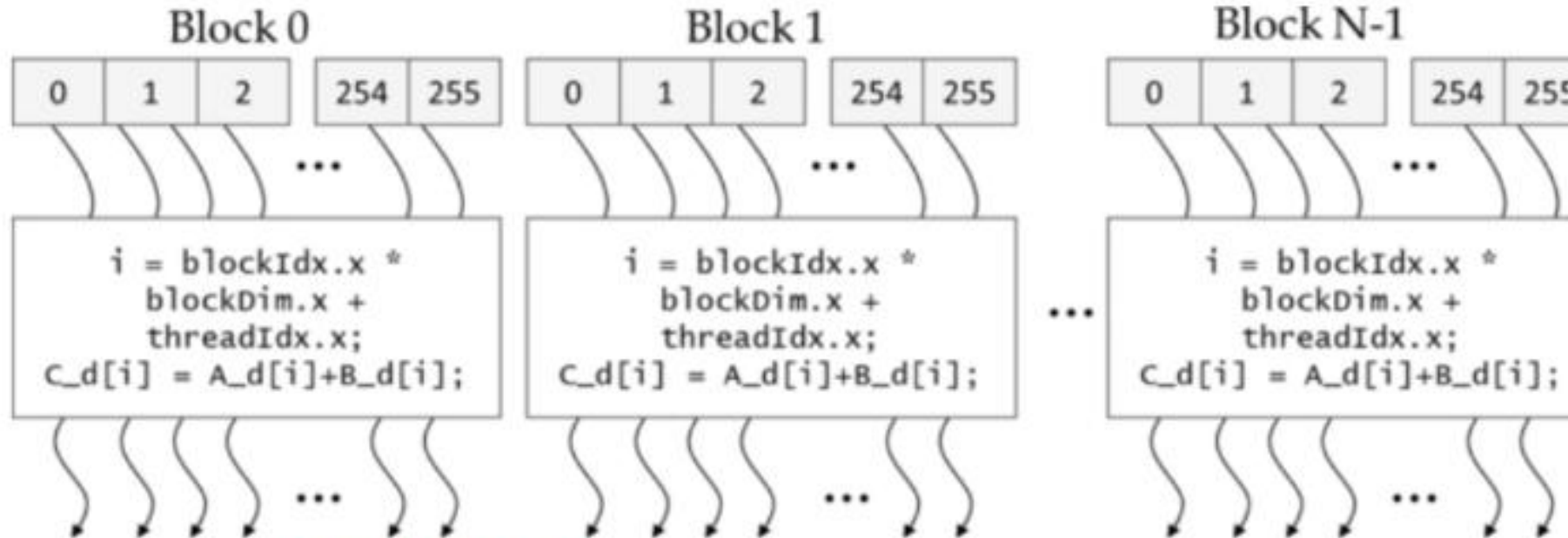


FIGURE 2.9 All threads in a grid execute the same kernel code.

- A kernel function specifies the code to be executed by GPU threads during a parallel phase. All threads run the same code (Single-Program Multiple-Data)

GPU Kernel: Grid, Block, Thread

- When a program's host code calls a kernel, the CUDA runtime system running on the host launches a grid of GPU threads
- Threads are organized as a two-level hierarchy.
 - A grid is organized as an array of thread blocks (also called just “blocks”)
 - All blocks of a grid have the same size.
 - Each block can contain up to 1024 threads on current systems.
- Built-in Variables to identify a thread:
 - `blockDim`: Number of threads in a block. A struct with three unsigned ints (x, y, and z) which can be used to organize threads of a block into one-, two-, or three-dimensional arrays. For one-dimensional arrays, only `blockDim.x` is used
 - `threadIdx`: Gives each thread a unique coordinate within a block. A struct with three unsigned ints x, y and z. For 1D, only `threadIdx.x` is used
 - `blockIdx`: Gives all blocks a unique coordinate in a grid. Also a struct with 3 unsigned ints x, y and z. For 1D, only `blockIdx.x` is used.

Vector Addition Kernel Function

Courtesy: PMPP Textbook

```
01  // Compute vector sum C = A + B
02  // Each thread performs one pair-wise addition
03  __global__
04  void vecAddKernel(float* A, float* B, float* C, int n) {
05      int i = threadIdx.x + blockDim.x * blockIdx.x;
06      if (i < n) {
07          C[i] = A[i] + B[i];
08      }
09  }
```

Why?

FIGURE 2.10 A simple vector addition kernel function.

- Each thread needs to identify its unique ID using built-in variables
 $\text{threadIdx.x} + \text{blockDim.x} * \text{blockIdx.x}$
- Note there is no “loop” like sequential program. Loop is replaced by the grid of threads; each thread running a loop iteration.
- Each thread in this kernel performs a single addition.

Declaring Functions in CUDA

Courtesy: PMPP Textbook

Qualifier Keyword	Callable From	Executed On	Executed By
<code>__host__</code> (default)	Host	Host	Caller host thread
<code>__global__</code>	Host (or Device)	Device	New grid of device threads
<code>__device__</code>	Device	Device	Caller device thread

FIGURE 2.11 CUDA C++ keywords for function declaration.

- `__host__` is the default and can be omitted
- `__global__` called from host/device and executed on device
- `__device__` called from device and executed on device, does not launch new threads

Calling Kernel Functions

Courtesy: PMPP Textbook

```
01  int vectAdd(float* A, float* B, float* C, int n) {  
02      // A_d, B_d, C_d allocations and copies omitted  
03      ...  
04      // Launch ceil(n/256) blocks of 256 threads each  
05      vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n);  
06  }
```

FIGURE 2.12 A host code example with a vector addition kernel call statement.

- Above code uses 1D grid with 1D blocks
- Each block is 256 threads, so we need $\text{ceil}(n/256.0)$ blocks to cover all vector elements
- Recall: Some threads may not perform any additions.

Vector Addition: Final Host Code

Courtesy: PMPP Textbook

```
01 void vecAdd(float* A, float* B, float* C, int n) {
02     float *A_d, *B_d, *C_d;
03     int size = n * sizeof(float);
04
05     cudaMalloc((void **) &A_d, size);
06     cudaMalloc((void **) &B_d, size);
07     cudaMalloc((void **) &C_d, size);
08
09     cudaMemcpy(A_d, A, size, cudaMemcpyHostToDevice);
10     cudaMemcpy(B_d, B, size, cudaMemcpyHostToDevice);
11
12     vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n);
13
14     cudaMemcpy(C, C_d, size, cudaMemcpyDeviceToHost);
15
16     cudaFree(A_d);
17     cudaFree(B_d);
18     cudaFree(C_d);
19 }
```

FIGURE 2.13 A vector addition kernel call statement.

- How do you expect performance to change with fewer threads (more operations per thread)?

Compiling CUDA C++ Programs

Courtesy: PMPP Textbook

- nvcc compiler used to compile CUDA C++ program
- Host code: straight C++ code; compiled with a standard C/C++ compiler; and runs as a CPU process
- Device code is marked with CUDA keywords that designate CUDA kernels and their associated device functions and data structures
- Device code is compiled into virtual binary files (called PTX files).
- PTX files are further compiled by a run-time component of nvcc into real object files to be executed on GPU device.

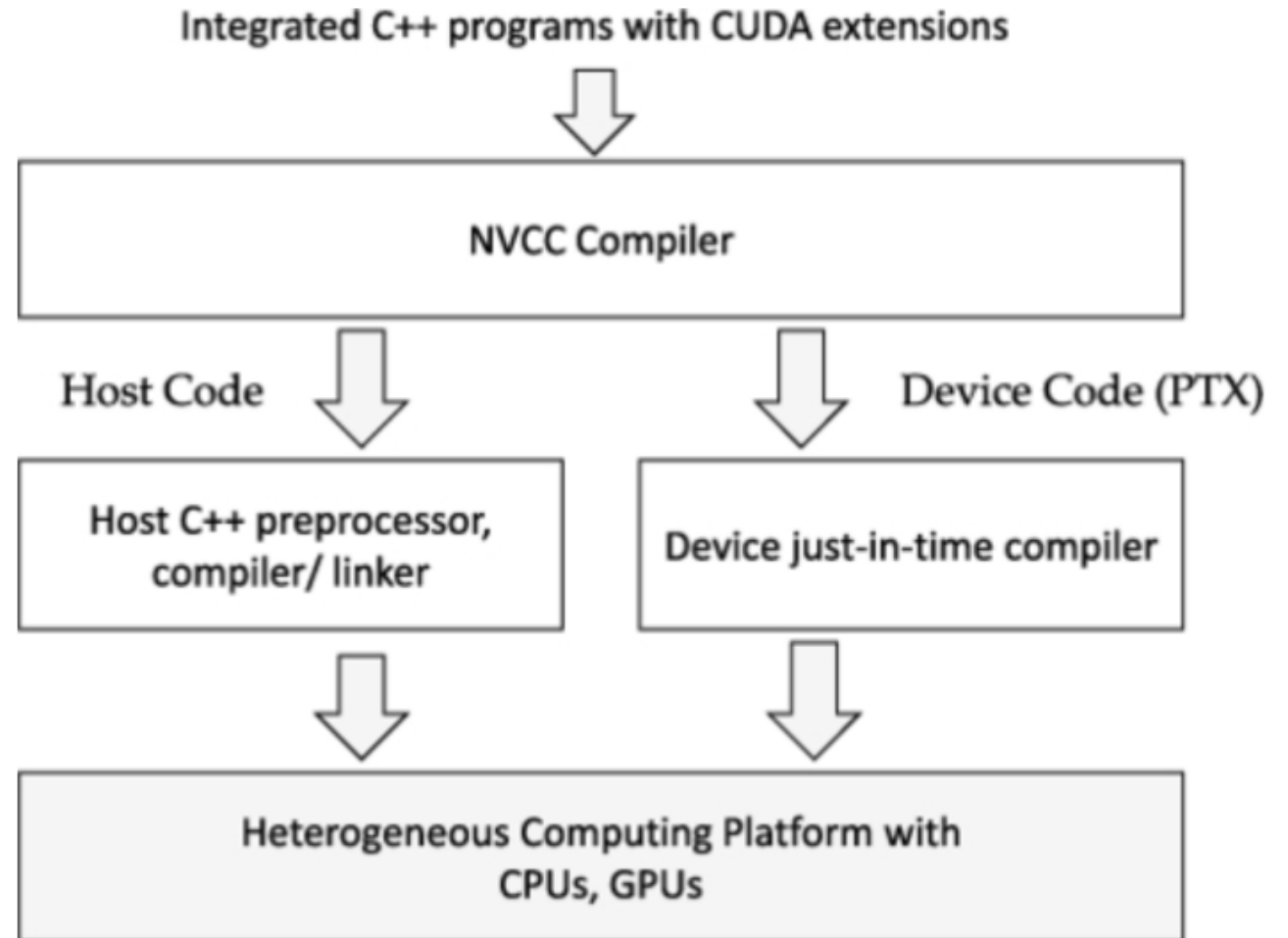


FIGURE 2.14 Overview of the compilation process of a CUDA C++ program.

Some Exercises (from Textbook)

Courtesy: PMPP Textbook

- If we want to use each thread in a grid to calculate one output element of a vector addition, what would be the expression for mapping the thread/block indices to the data index in our kernel function?

- a.** `i=threadIdx.x + threadIdx.y;`
- b.** `i=blockIdx.x + threadIdx.x;`
- c.** `i=blockIdx.x*blockDim.x + threadIdx.x;`
- d.** `i=blockIdx.x * threadIdx.x;`

Some Exercises (from Textbook)

Courtesy: PMPP Textbook

- Assume that we want to use each thread to calculate two adjacent elements of a vector addition. What would be the expression for mapping the thread/block indices to the data index of the first element to be processed by a thread?

a. `i=blockIdx.x*blockDim.x + threadIdx.x +2;`

b. `i=blockIdx.x*threadIdx.x*2;`

c. `i=(blockIdx.x*blockDim.x + threadIdx.x)*2;`

d. `i=blockIdx.x*blockDim.x*2 + threadIdx.x;`

Some Exercises (from Textbook)

Courtesy: PMPP Textbook

- Consider the CUDA kernel and the corresponding host function that calls it in Fig. 2.15.
 - a. What is the number of threads per block?
 - b. What is the number of threads in the grid?
 - c. What is the number of blocks in the grid?
 - d. What is the number of threads that execute the code on line 02?
 - e. What is the number of threads that execute the code on line 04?

```
01  __global__ void foo_kernel(float* a, float* b, unsigned int N){
02      unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
03      if(i < N) {
04          b[i] = 2.7f*a[i] - 4.3f;
05      }
06  }
07  void foo(float* a_d, float* b_d) {
08      unsigned int N = 200000;
09      foo_kernel <<< (N + 128 - 1)/128, 128 >>>(a_d, b_d, N);
10  }
```

FIGURE 2.15 CUDA code for Exercise Question 9.

Multi-Dimensional Grids and Data

Multi-Dimensional Grid Organization

- All threads in a grid execute the same kernel function. Threads rely on their coordinates, i.e., `threadIdx`, to distinguish themselves from others and identify the appropriate portion of the data to process.
- All threads in the same block share the same block index (`blockIdx`).
- Execution configuration parameters in a kernel call statement specify the dimensions of the grid (`gridDim`) and the dimensions of each block (`blockDim`)
- Example: Call to `vecAddKernel` with 32 blocks, 128 threads each:

```
dim3 dimGrid(32, 1, 1);
```

```
dim3 dimBlock(128, 1, 1);
```

```
vecAddKernel<<<dimGrid, dimBlock>>>(...);
```

- `dimGrid` and `dimBlock` can be any legal C++ variable name but need to be of type `dim3` (special case for 1D, `dim3` can be omitted like `vecAdd` example)
- Grid and block dimensions can be computed from other variables, for example if total number of vector elements is unknown in advance.

Limits on Grid and Block Dimensions in CUDA C++

- Allowed values of `gridDim.x` range from 1 to $2^{31}-1$
- Allowed values of `gridDim.y` and `gridDim.z` range from 1 to $2^{16}-1$ (65,535)
- All threads in a block share the same `blockIdx.x`, `blockIdx.y`, and `blockIdx.z` values.
- Among blocks, the `blockIdx.x` value ranges from 0 to `gridDim.x-1`, the `blockIdx.y` value ranges from 0 to `gridDim.y-1`, and the `blockIdx.z` value ranges from 0 to `gridDim.z-1`
- A block is a 3D array of threads. For 2D arrays, `blockDim.z` is set to 1. For 1D arrays, both `blockDim.z` and `blockDim.y` are set to 1
- A block has a maximum of 1024 threads, divided across 3 dimensions (product of three dimensions cannot exceed 1024). For example, `blockDim(16, 32, 2)` is valid while `(32, 32, 2)` is invalid

Example 3D Grid

Courtesy: PMPP Textbook

- Created using

```
dim3 dimGrid(2, 2, 1);  
dim3 dimBlock(4, 2, 2);  
KernelFunction<<<dimGrid, dimBlock>>>(...);
```
- Example: Block (1,0) has `blockIdx.y=1` and `blockIdx.x=0`
- Note that the ordering of the block and thread labels is such that highest dimension comes first (reverse ordering of that used in the code for setting configuration parameters where the lowest dimension comes first)

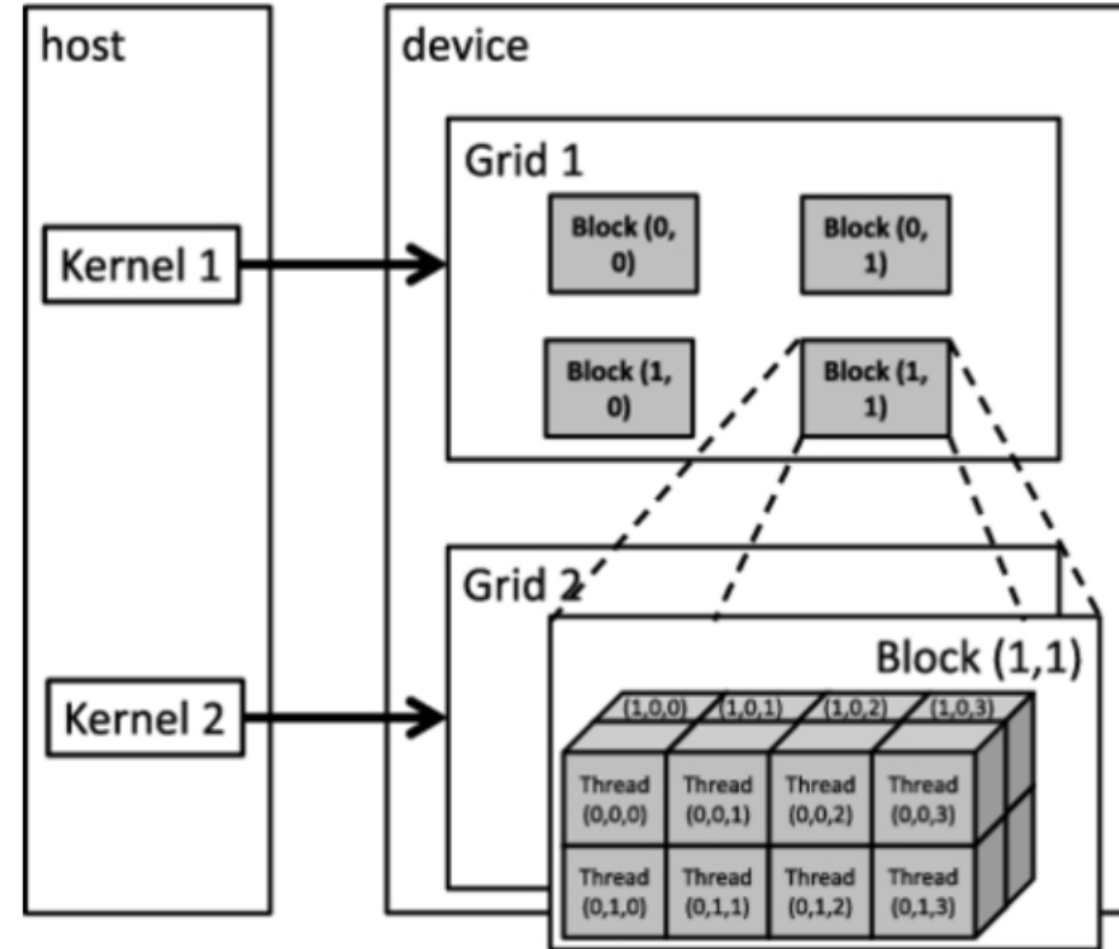


FIGURE 3.1 A multidimensional example of a CUDA grid organization.

Mapping Threads to MultiDimensional Data

- Choice depends on input data
 - Vectors: 1D is a better fit
 - 2D images, matrices: 2D is a better fit
 - 3D images: 3D is a better fit
- Example: Color to Grayscale conversion
 - Input Picture *Pin* is 62x76 pixels: 62 rows in vertical (y) dimension, 76 columns in horizontal (x) direction
 - Assume we use 2D blocks of 16x16 threads. So we need 4 blocks in y direction and 5 blocks in x direction
 - Each thread is assigned to work on a pixel depending on its threadIdx.
 - Vertical (row) coordinate is
$$\text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$$
 - Horizontal (column) coordinate is
$$\text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$
 - If needed (not for this problem), the plane (z) coordinate is
$$\text{blockIdx.z} * \text{blockDim.z} + \text{threadIdx.z}$$

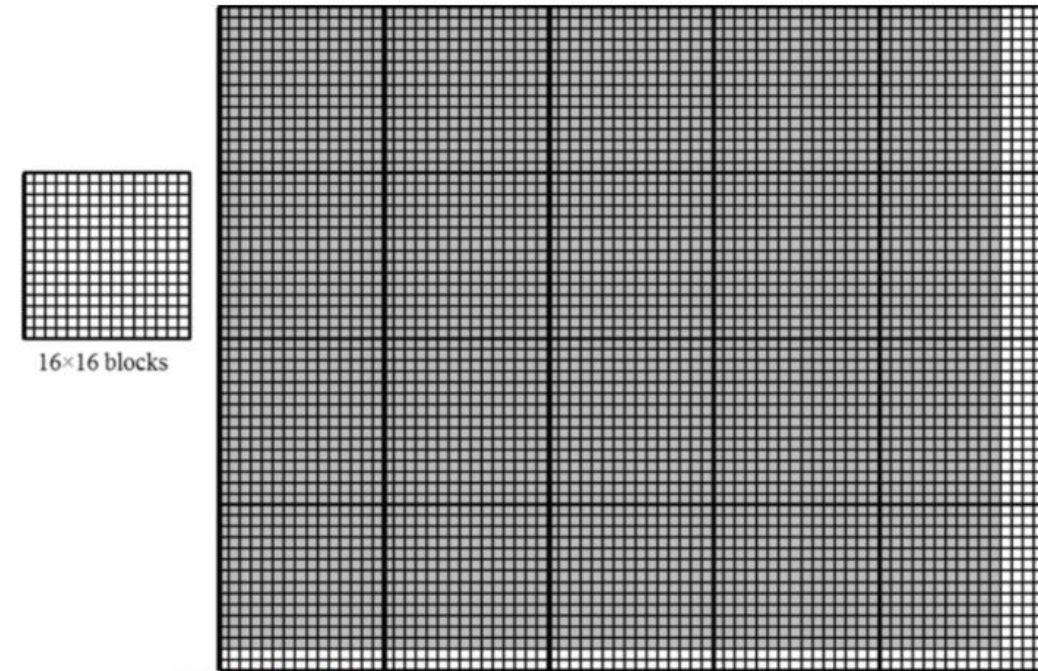


FIGURE 3.2 Using a 2D thread grid to process a 62×76 picture *Pin*.

Courtesy: PMPP Textbook

Mapping Data to Device Memory

- Typically data is mapped to memory in row-major order: All data of first row is stored first, then all data of second row, etc. This results in an equivalent linear (1D) array

➤ Column-major order stores all data of first column, then all data of second column etc.

- Example: Color to Grayscale conversion

➤ Vertical (row) coordinate is

`blockIdx.y * blockDim.y + threadIdx.y`

➤ Horizontal (column) coordinate is

`blockIdx.x * blockDim.x + threadIdx.x`

➤ If needed (not for this problem), the plane (z) coordinate is

`blockIdx.z * blockDim.z + threadIdx.z`

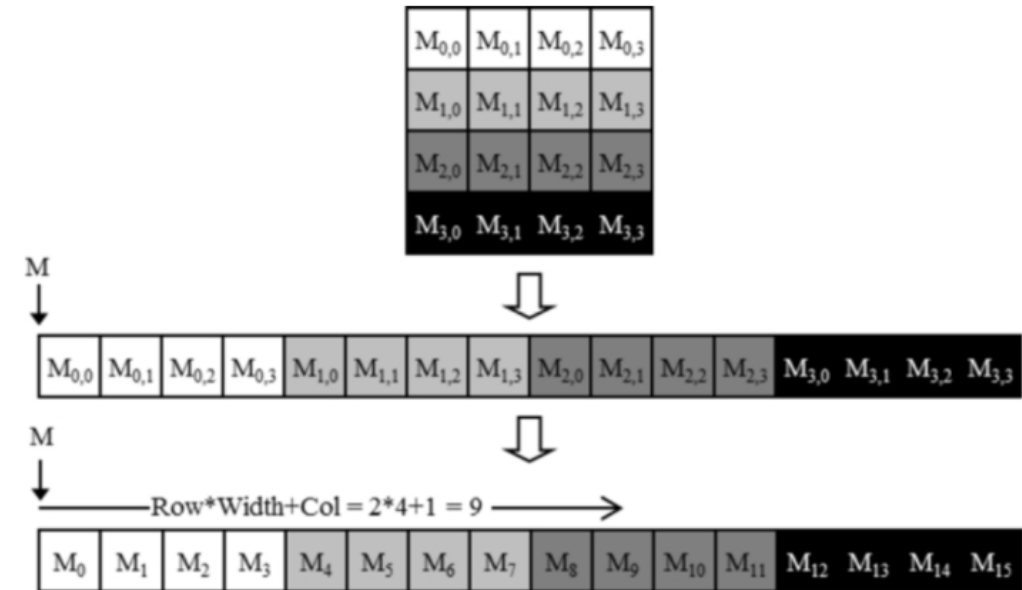


FIGURE 3.3 Row-major layout for a 2D C++ array. The result is an equivalent 1D array accessed by an index expression $j * \text{Width} + i$ for an element that is in the j^{th} row and i^{th} column of an array of *Width* elements in each row.

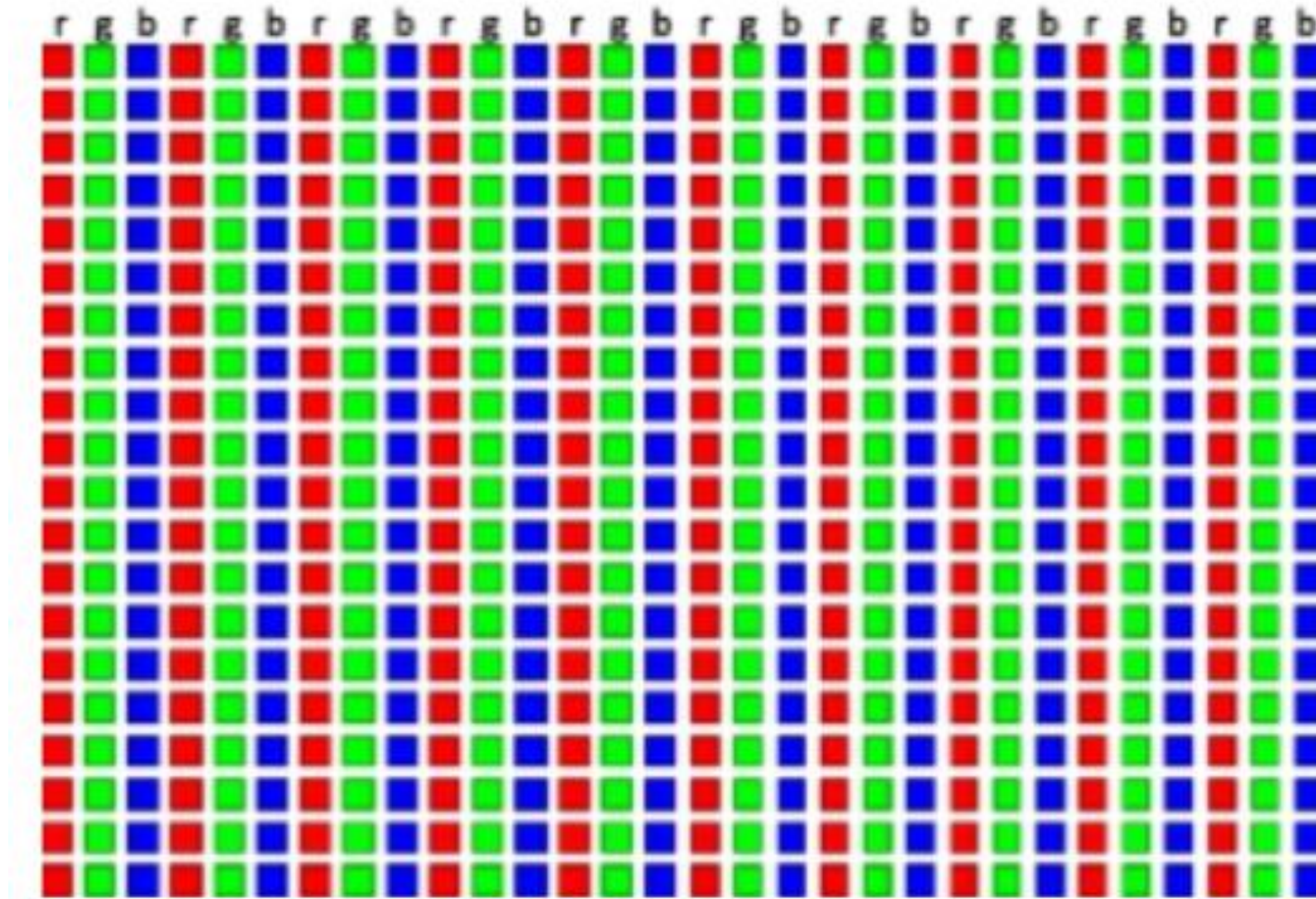
Courtesy: PMPP Textbook

Color to Grayscale Conversion: Calling Kernel

- 16x16 blocks
- Grid dimensions based on number of rows or columns (m rows, n columns)
- Calling the kernel:

```
dim3 dimGrid(ceil(m/16.0), ceil(n/16.0), 1);  
dim3 dimBlock(16, 16, 1);  
colorToGrayscaleConversion<<<dimGrid, dimBlock>>>(Pin_d, Pout_d, m, n);
```

Input Color is 3 bytes per pixel



Courtesy: PMPP Textbook

Color to Grayscale Conversion Kernel

```
01 // The input image is encoded as unsigned chars [0, 255]
02 // Each pixel is 3 consecutive chars for the 3 channels (RGB)
03 __global__
04 void colorToGrayscaleConversion(unsigned char * Pout,
05                                unsigned char * Pin, int width, int height) {
06     int col = blockIdx.x*blockDim.x + threadIdx.x;
07     int row = blockIdx.y*blockDim.y + threadIdx.y;
08     if (col < width && row < height) {
09         // Get 1D offset for the grayscale image
10         int grayOffset = row*width + col;
11         // One can think of the RGB image having CHANNEL
12         // times more columns than the gray scale image
13         int rgbOffset = grayOffset*CHANNELS;
14         unsigned char r = Pin[rgbOffset]; // Red value
15         unsigned char g = Pin[rgbOffset + 1]; // Green value
16         unsigned char b = Pin[rgbOffset + 2]; // Blue value
17         // Perform the rescaling and store it
18         // We multiply by floating point constants
19         Pout[grayOffset] = (unsigned char) 0.21f*r+0.71f*g+0.07f*b;
20     }
21 }
```

Courtesy: PMPP Textbook

FIGURE 3.4 Source code of
colorToGrayscaleConversion with
2D thread mapping to data.

Boundary Conditions

- Area 1: All threads have an assigned pixel to process
- Area 2: row values are always within range, but col values of some of them exceed 76. So 12 threads in each row will process pixels; while the remaining 4 threads in each row will fail
- Area 3: col values are always within range, but row values for the last two rows exceed 62. So 14 threads in each column will process pixels while the remaining two will fail
- Area 4: 4 threads in each of the top 14 rows will find their col values out of range. The entire bottom two rows will find their row values out of range. Overall, only $14 \times 12 = 168$ of the 256 threads in that will process pixels

Courtesy: PMPP Textbook

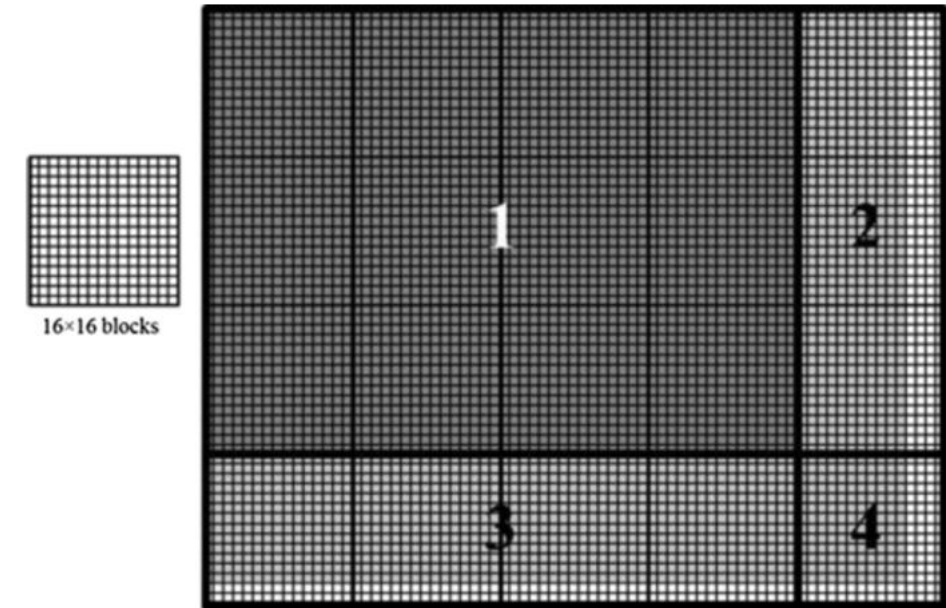


FIGURE 3.5 Covering a 76×62 picture with 16×16 blocks.

Class Announcements

- **HW1: Due Monday Sep 14**

- Online quiz intended to get you to assess whether you have the right background for this course. Everyone is expected to get 100% 😊
- You can take up to 3 times. Latest score will be used.

- **HW2: Released next week. Due Tuesday Sep 22**

- **This week's reading:**

- LLNL Parallel Computing Tutorial (skim).
- PMPP Chapters 1-2, 3.1-3.2

- **Next week's reading:**

- PMPP Chapters 3.3-3.5, 4